Phase 1 — Conception

Hotel Booking Chatbot · Aurora Grand Hotel
IU Project: AI Use Case (CSEMAIPAIUC01) · Task 1
Student-Name · Matriculation 12345678

Value proposition

For leisure and business travellers who want to book a hotel room in under a minute, **dissatisfied with** long multi-step web forms that demand account creation and hide availability behind filters, **due to** the unmet need for a fast, mobile-friendly, conversational booking flow, **Aurora Grand Hotel offers** Aria, a conversational booking assistant, **that provides** a single-channel chat interface, instant confirmation with a reference number, and 24/7 availability with no account required.

Aims and audience

Aria's goal is to capture the minimum information needed to create a valid room reservation through natural-language dialogue. The minimum viable segment (MVS) is English-speaking travellers comfortable with chat-style interfaces (~70% of online hotel bookings originate on mobile). The minimum viable product (MVP) therefore reduces the form to six slots and produces a booking reference on confirmation.

Structure and process rationale

A **slot-filling state machine** was chosen over a free-form LLM approach because the booking domain is narrow and closed, correctness is paramount (a hallucinated reservation is unacceptable), and deterministic behaviour makes the system trivially auditable for a course grader. The bot fills the slots *name* → *dates* → *guests* → *breakfast* → *payment* → *confirm*; the user may volunteer information out of order, and the classifier falls back to a graceful hint if an entity cannot be parsed.

Chosen frameworks and tools

Layer	Choice	Reasoning
Runtime	Python 3.10+	Ubiquitous, runs on the tutor's machine with no additional toolchain.
Web framework	Flask 3	Smallest possible footprint for a single-page chat demo.
Intent NLU	Bag-of-words + cosine similarity	Transparent, zero extra dependencies, easy to explain in a viva.
Entities	Regex + heuristics	Deterministic for the fixed domain; robust on tiny training data.
Persistence	JSON file	No DB service needed; grader can inspect bookings directly.
Frontend	Vanilla HTML/CSS/JS	No build step — open the file and read it.

Trade-offs vs. alternative stacks

Rasa was the course's suggested tool but was ruled out for this prototype: it fails to install on Python 3.14 (dependency chain still requires older Python) and its production focus — custom actions server, tracker store, NLU pipeline — is overkill for a six-slot form. **Dialogflow** was also considered but rejected because it requires a Google Cloud account, outbound internet connectivity at grading time, and exports dialogue logic to an external service.

Mandatory and optional information

Mandatory (per assignment spec): guest name · stay period (check-in and check-out) · number of guests.
Optional (supported): breakfast preference · payment method (credit card, debit card, or pay at hotel).

UML diagrams

Three diagrams document the system from complementary angles: components, runtime sequence, and dialog state.

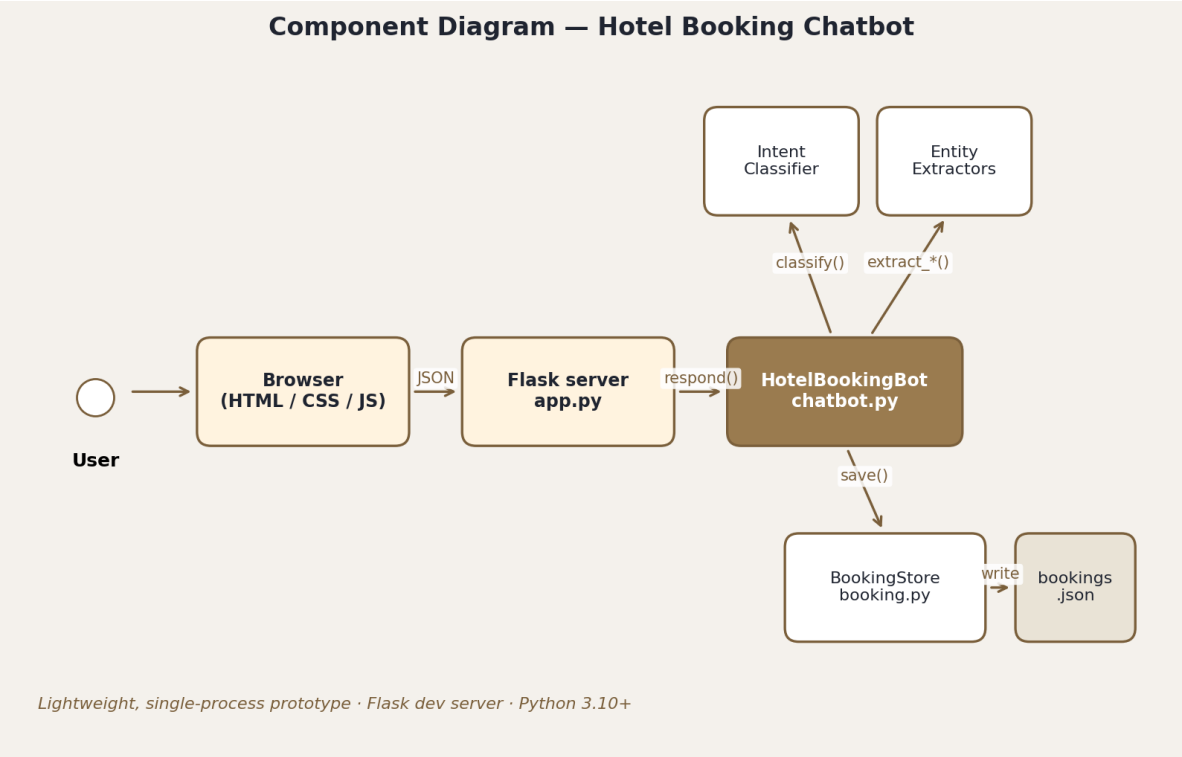

Figure 1 — Component diagram. The browser speaks JSON to Flask; the bot orchestrates an intent classifier, entity extractors, and a JSON booking store.

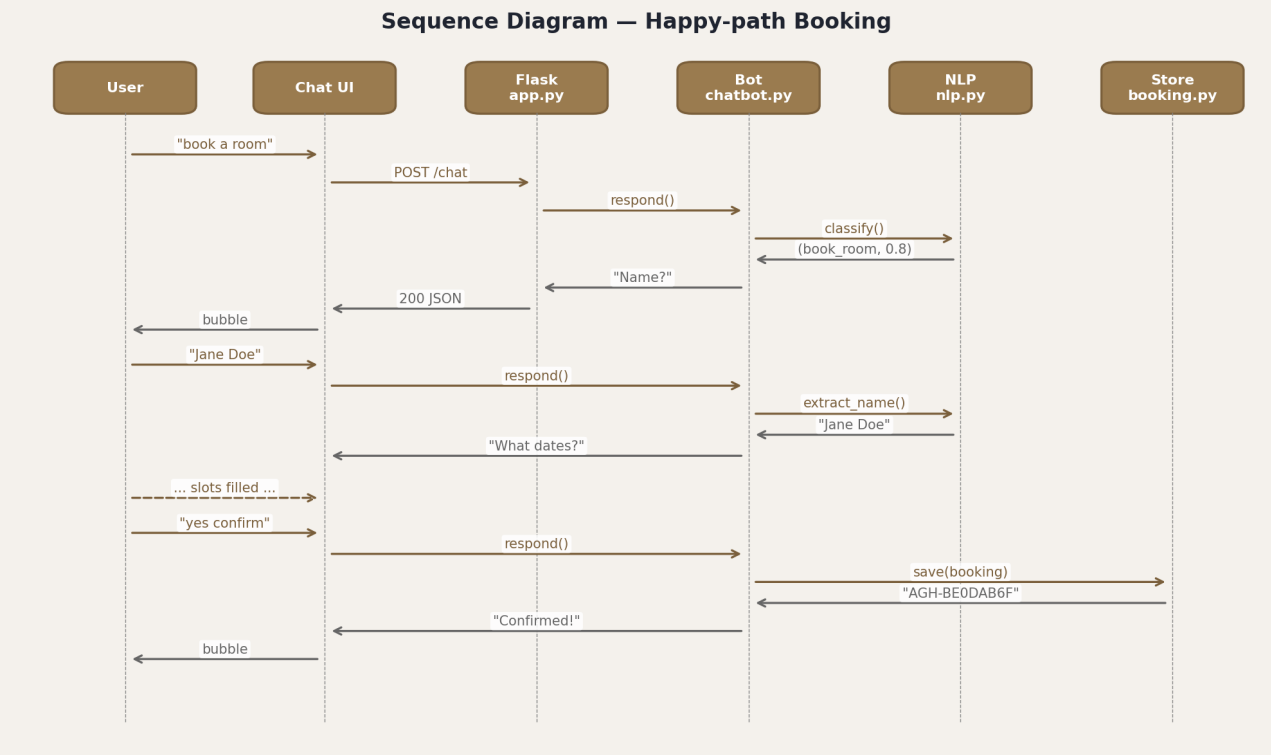

Figure 2 — Sequence diagram. Happy-path booking: greet → name → dates → … → confirm; each turn is one round-trip through Flask and the dialog manager.

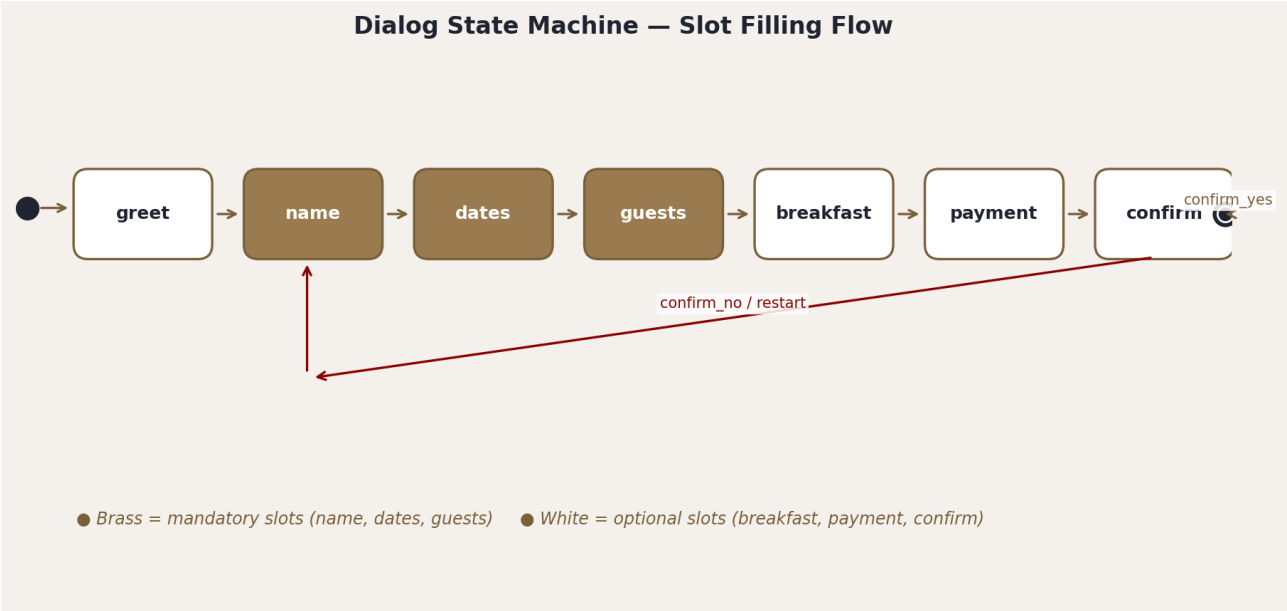

Figure 3 — State diagram. Slot-filling state machine; mandatory slots in brass, optional slots in white. 'confirm_no' loops back to the name step.